Deep Learning for NLP - Lab 2

Task 1 — Byte Pair Encoding (BPE)

(0.20p) Implement Byte Pair Encoding. Use the small corpus discussed in class as test example.

```
low
new
lowest
newest
widest
lower
```

- Start from word forms split into characters, using an end-of-word marker if you choose to follow the lecture notation.
- Count all adjacent symbol pairs.
- Find the most frequent pair and merge it into a new symbol.
- Repeat the process for at least 5 merge operations.
- After each merge, display the selected pair and the updated corpus/vocabulary.
- Apply the learned merges, in the same order, to at least one unseen word.

Test word: slowest

Useful resource: Hugging Face — Byte-Pair Encoding tokenization

Task 2 — Generalized Romanian N-gram Language Model

(0.40p) Implement a generalized N-gram language model for Romanian using a corpus you collect from the Web.

- Collect a Romanian corpus containing at least 1,000 words. Record the source URL(s).
- Preprocess and tokenize the corpus consistently.
- Allow the user to choose N, for example N=1 (unigram), N=2 (bigram), N=3 (trigram).
- Compute N-gram counts and the corresponding conditional probabilities.
- Implement one smoothing technique of your choice: Laplace/add-one, interpolation, or another technique if properly explained.
- Report source, corpus statistics and the most frequent N-grams.

Your program should report at least:

```
number of tokens
vocabulary size |V|
number of distinct N-grams
```

most frequent N-grams

Smoothing check: select at least one N-gram and compare its probability before and after smoothing.

Task 3 — Probability of a New Romanian Sentence

(0.20p) Given a Romanian sentence entered by the user, compute its probability using the N-gram model developed in Task 2.

- Tokenize the input sentence using exactly the same preprocessing/tokenization policy as for the training corpus.
- Include sentence-boundary markers <s> and </s> when appropriate.
- Display the individual conditional probabilities that contribute to the sentence probability.
- Compute the final sentence probability.
- Also compute the log-probability to avoid numerical underflow on longer sentences.

Task 4 — Compare with a Pretrained Neural Language Model

(0.20p) Use a pretrained causal language model to continue a short input sequence.

- Choose a pretrained causal language model (GPT-style or another left-to-right autoregressive model).
- Provide an input sequence of at least 4 words.
- Tokenize the input with the model tokenizer.
- Generate the next two tokens and decode them back to text.
- Report the generated continuation.
- Briefly compare what the neural model predicts with what your N-gram model from Task 2 would predict for the same or a comparable context.

Useful resource: Hugging Face — Causal Language Modeling

Submission

- Submit one Jupyter/Colab notebook or Python project containing all four tasks.
- Include the Romanian corpus source URL(s) and clearly state your preprocessing choices. Report the required corpus statistics and N-grams.
- Include outputs for the required examples, not only code.
- For every major result, add 1–3 sentences of interpretation.
- Your code should be reproducible from a fresh runtime, including package installation instructions when necessary.